

Advanced JVM & Production Engineering Deep Dive

1. GC Log Sample + Real Analysis Walkthrough

Sample G1GC Log Snippet:

```
[GC pause (G1 Evacuation Pause) (young), 0.0456789 secs]
  [Eden: 512M(512M)->0B(512M) Survivors: 32M->64M Heap: 1024M(2048M)->600M(2048M)]
```

```
[Full GC (Allocation Failure) 2048M->1200M(2048M), 3.4567890 secs]
```

Analysis Steps:

1. Identify GC type (Young vs Full).
2. Check pause time (STW impact).
3. Observe heap occupancy before/after.
4. If Full GC frequent → Old Gen pressure or memory leak.

Common Root Causes:

- High promotion rate
- Survivor space overflow
- Large (humongous) object allocation
- Retained objects via static collections

Resolution Strategy:

- Increase heap (-Xmx)
- Tune G1 regions (-XX:G1HeapRegionSize)
- Enable String Deduplication
- Analyze heap dump via MAT
- Switch to ZGC for ultra-low latency (Java 15+)

Manager-Level Insight:

Full GC = Stop-The-World → latency spikes in production SLAs.

2. JMM Deep Dive with CPU-Level Explanation

Problem:

Modern CPUs use multi-level caching (L1, L2, L3). Threads may read stale values.

Without JMM:

CPU may reorder instructions for optimization.

Example:

```
int a = 1;
```

```
int b = 2;
```

CPU could reorder writes unless constrained.

JMM Solutions:

- Memory barriers (LoadLoad, StoreStore, StoreLoad)
- Happens-before guarantees

volatile:

Adds memory fence instructions at bytecode/JIT level.

Ensures write-through to main memory.

CAS (Compare-And-Swap):

Implemented via CPU atomic instructions (cmpxchg).

Used in AtomicInteger, ConcurrentHashMap.

False Sharing:

Two threads modify adjacent variables → same cache line → performance degradation.

Solution: Padding or @Contended (Java 8+).

Key Concepts:

- Visibility
- Atomicity
- Ordering
- Sequential consistency (only guaranteed with proper synchronization)

3. Spring Boot Memory Behavior + Docker/K8s Tuning

Spring Boot Memory Breakdown:

- Heap (Objects, Beans)
- Metaspace (Class metadata)
- Thread stacks
- Direct buffers (Netty)
- Code cache

Common Production Issue:

Container memory limit = 1GB

JVM default heap may assume host memory (8GB)

→ OOMKilled by Kubernetes

Solution:

Use container-aware JVM (Java 10+)

Flags:

-XX:+UseContainerSupport

-XX:MaxRAMPercentage=75.0

Recommended for K8s:

- Set Xms = Xmx (avoid resizing cost)
- Tune Xss to reduce thread stack memory
- Monitor via Micrometer + Prometheus

Memory Formula (Container):

Total Memory =

Heap + Metaspace + ThreadStacks + DirectMemory + OS overhead

Example:

Heap: 512M

Metaspace: 128M

Threads (200 * 512k): 100M

Direct: 128M

Total ≈ 868M

Always leave 10-15% headroom.

GC Recommendation:

Low latency → ZGC / Shenandoah (Java 17+)

Throughput → G1GC

4. 40 Strict Manager-Level Cross-Examination Questions

1. Explain G1GC region-based design.
2. What is a humongous object in G1?
3. Difference between Minor, Major, Full GC?
4. What triggers Full GC in G1?
5. How does ZGC achieve sub-10ms pause?
6. What is colored pointer in ZGC?
7. Why is CMS deprecated?
8. How does JIT decide hot methods?
9. What is Tiered Compilation?
10. What is Safepoint bias?
11. How to detect memory leak in production?
12. Explain escape analysis.
13. What is TLAB?
14. Difference between StackOverflowError and OOM?
15. What causes Metaspace OOM?
16. How classloader leak happens?
17. Explain biased locking.
18. When does lock inflation occur?
19. How does ConcurrentHashMap avoid locking entire map?
20. What is fork-join pool?
21. How CompletableFuture uses it?
22. What is backpressure in reactive systems?
23. What is DirectByteBuffer?
24. When does GC not free memory?
25. Explain GC roots.
26. What is ReferenceQueue?
27. Soft vs Weak vs Phantom reference?
28. Explain CPU cache coherence protocol.
29. What is MESI?
30. How thread context switching impacts latency?
31. What is pinned object?
32. What is GC overhead limit exceeded?
33. What is memory fragmentation?
34. What is compressed oops?
35. When disable compressed oops?
36. How to read JFR flame graph?
37. Explain allocation rate vs promotion rate.
38. What is coordinated omission?
39. How to tune JVM for high-frequency trading?
40. Design JVM tuning strategy for 10k RPS microservice.

Expectation:

Candidate must explain reasoning, trade-offs, and production decisions.